Biogeochemical Modelling of Lakes, Sediments and Oceans
701-1338-00L 2026

Numerical Modeling of Ordinary Differential Equations (ODEs)

Aaron Wienkers, awienkers@ethz.ch

Original Slide Credit: Abolfazl Irani Rahaghi

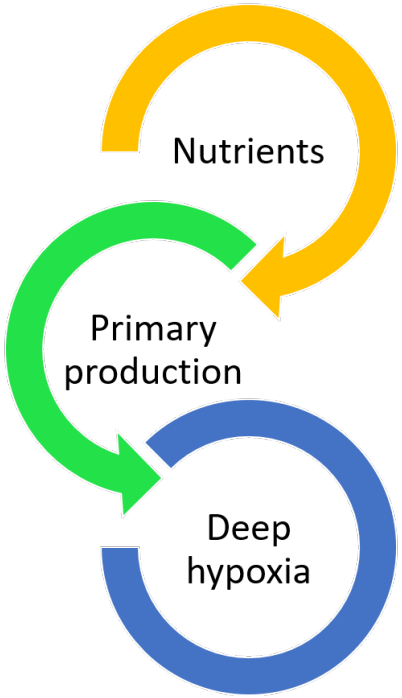
Download

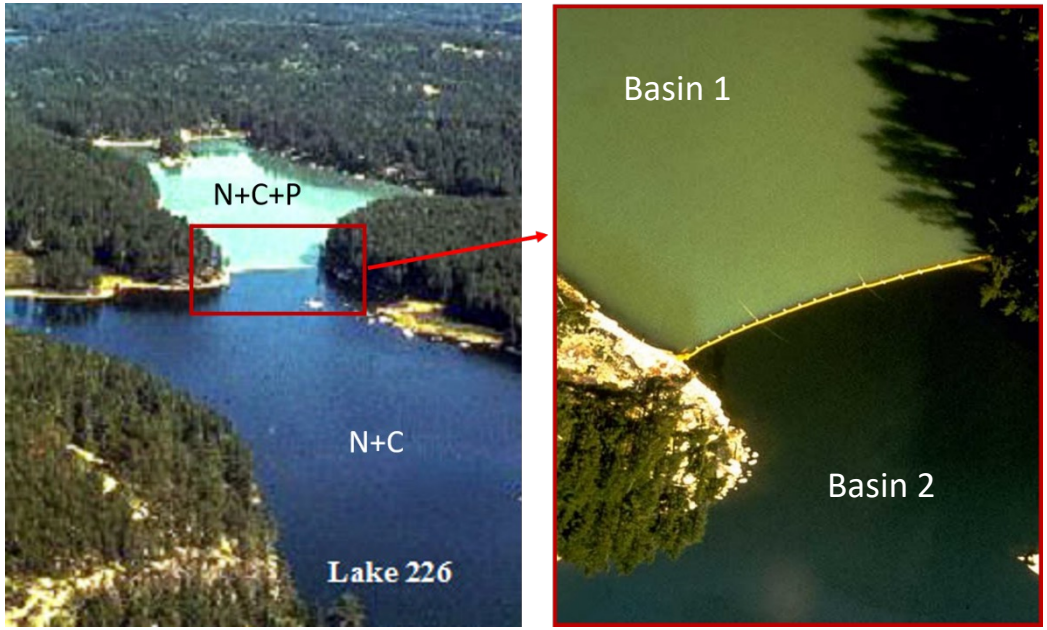
Examples and Exercises:
Numerical_methods_Part_I.ipynb

Data:
Sempachsee.csv

Solution:
Numerical_methods_Part_I_solutions.ipynb

Eutrophication issue in water bodies



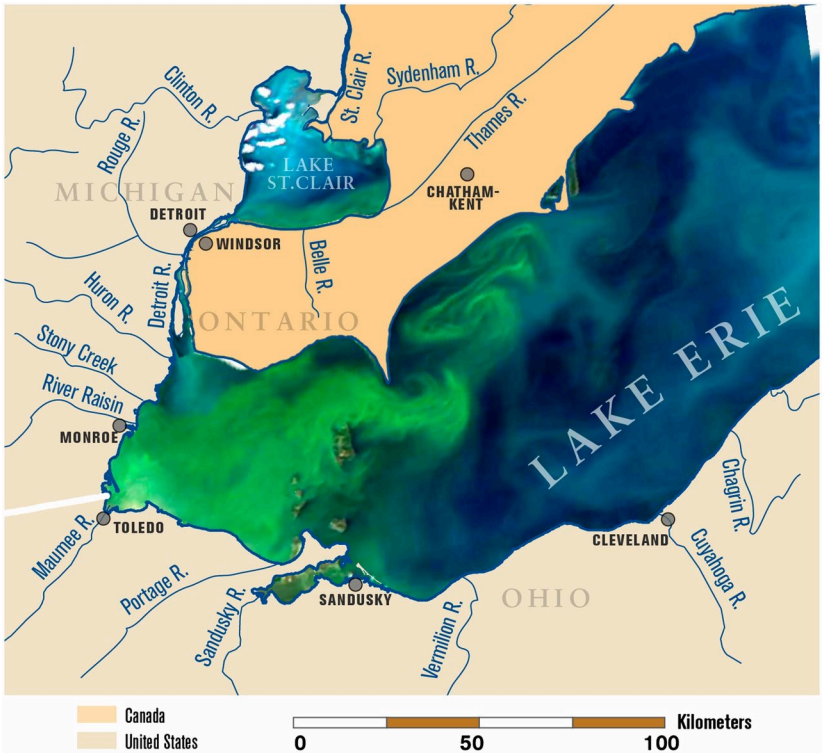


- Eutrophication: Increased growth of algae (microorganisms) due to accumulated nutrients in a water body
- Adverse impacts: oxygen depletion in the deep water, dead zones, and fish kills

- Basin 1, fertilized with phosphorus, nitrogen, and carbon, was covered by an algal bloom within 2 months.
- No increase in algae were observed in Basin 2, which received similar quantities of nitrogen and carbon, but no phosphorus.

Source: Schindler (1974) Science https://doi.org/10.1126/science.184.4139.897

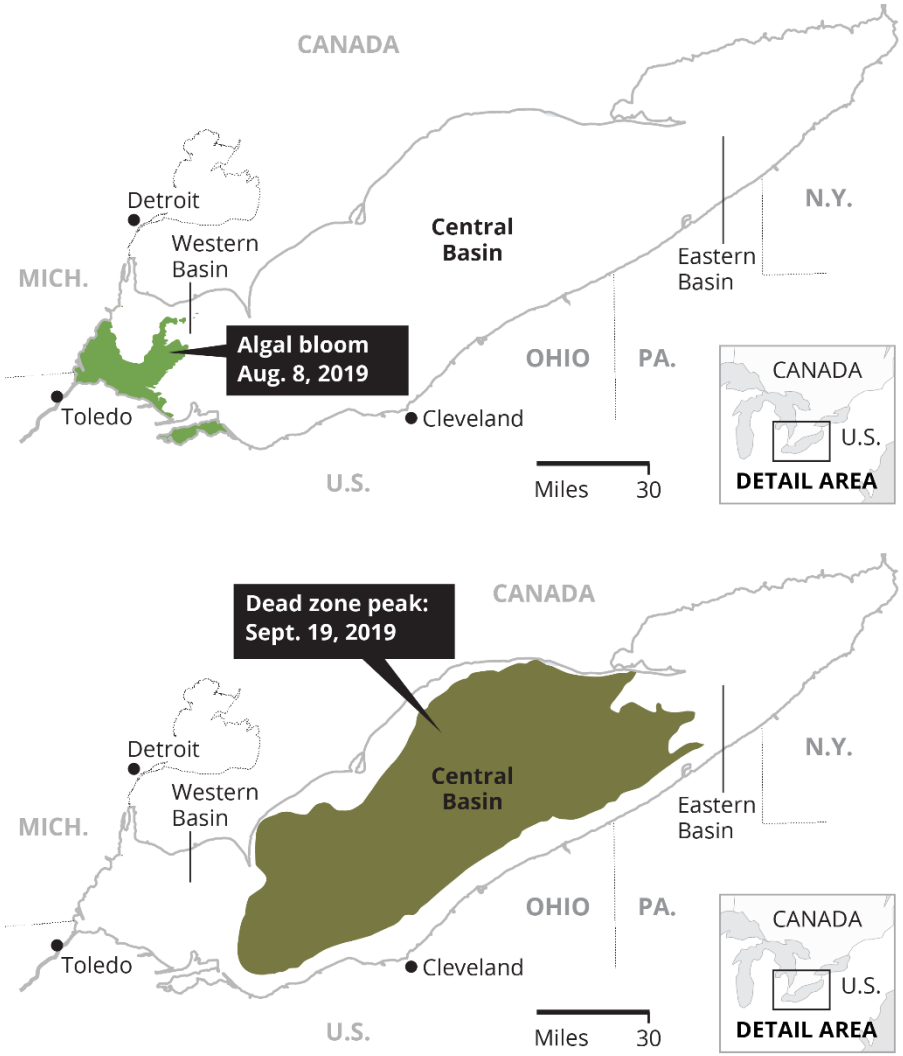
Lake eutrophication: Lake Erie (USA/Canda)



Lake Erie - Satellite image 2011
Source: Michalack et al (2013) PNAS
https://doi.org/10.1073/pnas.1216006110

The 3D hydrodynamic modeling based on Princeton Ocean Model

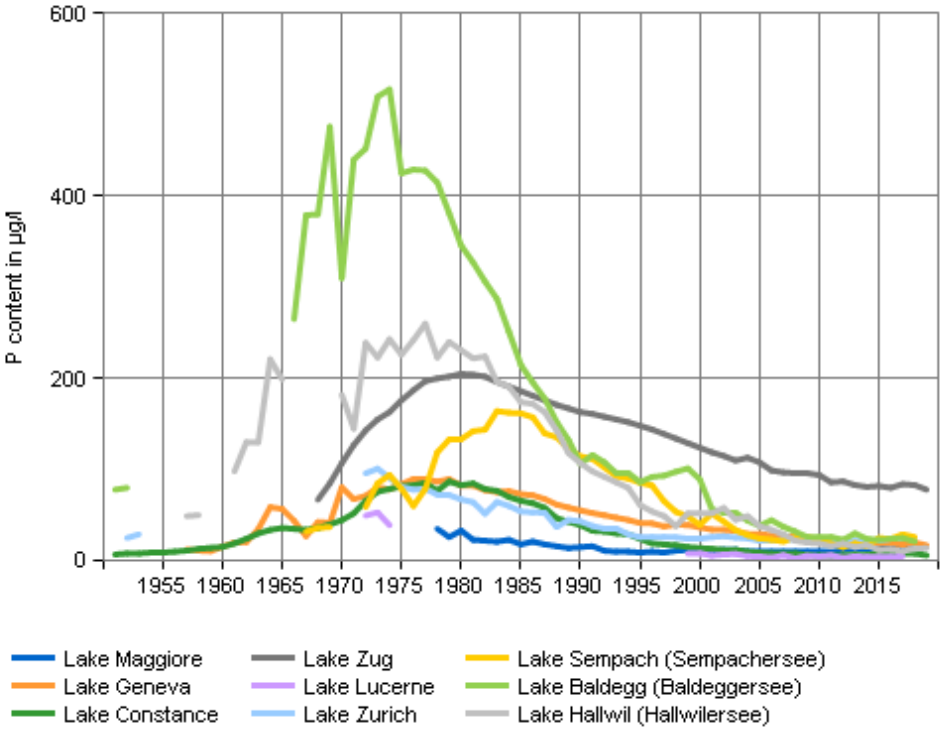
http://www.ccpo.odu.edu/POMWEB/index.html



Source: NOAA + Chicago Tribune
https://graphics.chicagotribune.com/lake-erie-charts/blurb.html

Lake management response in Switzerland

Reduction in phosphorus loading since the 1980s



Since the 1980s, the water quality of Swiss lakes has improved considerably (primarily due to the development of wastewater treatment and the 1985 ban on phosphate in laundry detergents). However, some lakes still demonstrate high phosphate levels and so are eutrophic (over-enriched with nutrients).

Source: BAFU https://www.bafu.admin.ch/bafu/en/home/topics/water/info-specialists/state-of-waterbodies/state-of-lakes/water-quality-in-lakes.html/

Abb 2a und b: Greifensee 10.8.2011; *Microcystis aeruginosa*

Abb 3a und b: Lützelsee 28.8.2018; *Aphanizonemon flos-aquae*

Abb 4a und b: Ruderboot Greifensee 17.11.2020; *Gomphosphaeria sp.*, Fotos Klemens Rosin

Abb 5a und b: Hüttnersee 27.8.2021 und 6.9.2021; *Microcystis aeruginosa*

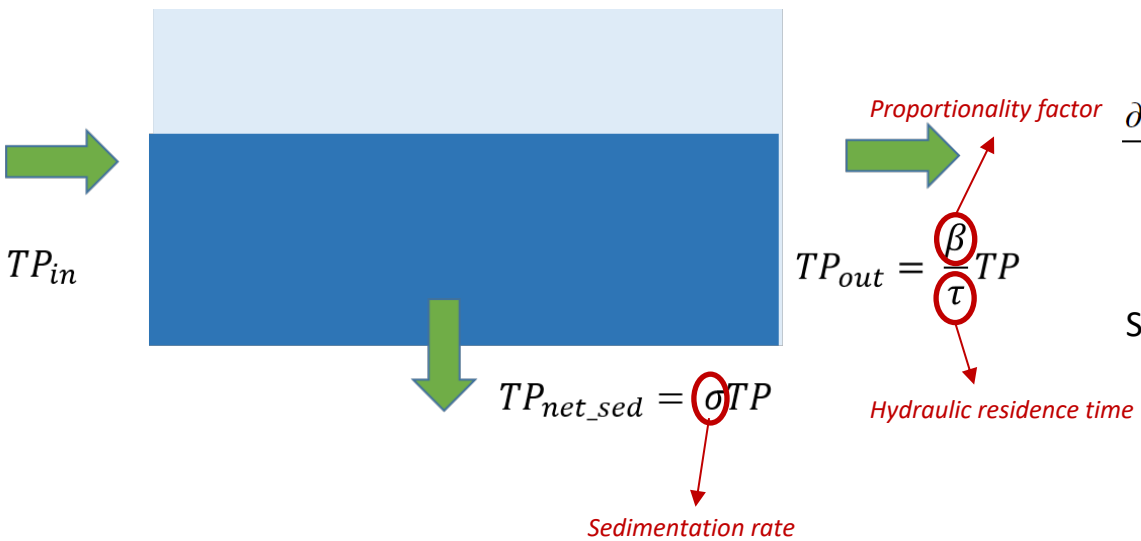
Source: https://www.zh.ch/content/dam/zhweb/bilder-dokumente/themen/umwelt-tiere/wasser-gewaesser/gewaesserqualitaet/merkblatt_blaualgen.pdf

Lake management response in Switzerland…

- **What is the maximal acceptable phosphorus loading?**
- **How quickly will the system adapt to a change in phosphorus loading?**

By law, lake water $[DO]$> 4 mg L^{-1} ⟹ corresponding $[TP]$ target is generally ~ 10 μg L^{-1}

Method ⟹ Vollenweider's phosphorus mass balance model



$$V \frac{\partial TP_{lake}}{\partial t} = TP_{in} - TP_{out} - TP_{sed}$$

$$\frac{\partial TP_{lake}}{\partial t} = \left(\frac{1}{V}\right) TP_{in} - \left(\frac{\beta}{\tau} + \sigma\right) TP_{lake}$$

Steady state solution:

$$TP_{in} = V.TP\left(\frac{\beta}{\tau} + \sigma\right)$$

Vollenweider RA (1976) Advances in defining critical loading levels for phosphorus in lake eutrophication. Mem. Ist. Ital. Idrobiol. 33:53–83

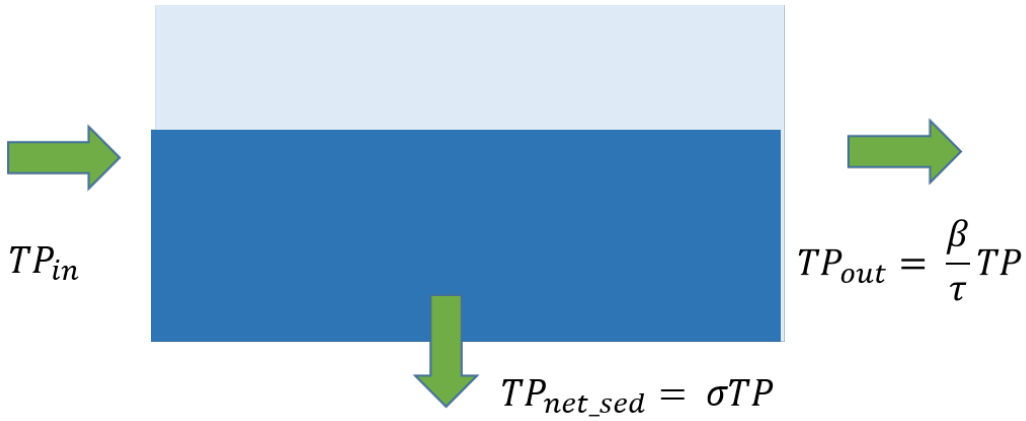
Example 1: Total phosphorus variation in Lake Sempach

- Read and plot the TP data for Lake Sempach (./Sempachsee.csv)
- Write a function to estimate TP variation based on Vollenweider's phosphorus mass balance model
- Compare the measured and modeled TP for Lake Sempach
- Calculate the steady state solution

Check the corresponding section in:
Numerical_methods_Part_I.ipynb

Source: BAFU https://www.bafu.admin.ch/dam/bafu/de/dokumente/wasser/fachinfo-daten/Wasserqualit%C3%A4t_Seen_Sempachersee_-_P072-1230.pdf.download.pdf/Wasserqualit%C3%A4t_Seen_Sempachersee_-_P072-1230.pdf

Model simplification



$$V \frac{\partial TP_{lake}}{\partial t} = TP_{in} - TP_{out} - TP_{sed}$$

$$\frac{\partial TP_{lake}}{\partial t} = \left(\frac{1}{V}\right) TP_{in} - \left(\frac{\beta}{\tau} + \sigma\right) TP_{lake}$$

$$\left.\begin{aligned} TP_{lake} &= C \\ TP_{in} &= I \\ \frac{\beta}{\tau} + \sigma &= K \end{aligned}\right\} \Rightarrow \frac{\partial C}{\partial t} = \frac{I}{V} - KC$$

A simple forward solver in time

For a small time Δt:

$$\frac{\partial C}{\partial t} \cong \frac{\Delta C}{\Delta t} \xrightarrow[\text{assuming a constant } \Delta t]{\text{At time step } \mathbf{n} \text{ and}} \frac{\partial C^n}{\partial t} \cong \frac{C^{n+1} - C^n}{\Delta t}$$

$$\Longrightarrow \quad \frac{C^{n+1} - C^n}{\Delta t} = \frac{I}{V} - K C^n$$

assuming to be constant

$$\Longrightarrow \quad C^{n+1} = (1 - K\Delta t)C^n + \frac{I}{V}\Delta t$$

Goal of this course

- To understand the concept of finite difference methods and some numerical schemes
- To understand how to transform an algorithm to a "well-designed" computer code
- To understand how to test (verify) the code
- To understand the potential artifacts in simulation results
- A broader perspective: to avoid being a clueless operator while using a modeling code/software

Some references for this section of the course:

- https://hplgit.github.io/
- https://www.springer.com/gp/book/9780387236193
- and many more that you can find

Systems governed by a simple ODE

$$\frac{dC(t)}{dt} = -kC(t)$$

- nuclear decay
- chemical reactions
- heating and cooling (Newtonian)
- finances
- SIR (Susceptible-Infected-Removed) model
- predator - prey model
- any process with exponential growth or decay

Analytical (exact) solution $\Longrightarrow$ $C(t) = C(0)e^{-kt}$

How to formulate a finite difference scheme?

- ## Step 0: Problem formulation

  Not the scope of this lecture, but it's probably the most important step.

$$\frac{dC(t)}{dt} = -kC(t)$$

- ## Step 1: Domain discretization

  We introduce a finite mesh that will approximate the continuous domain. Solution of the model will be expressed at these time points.

$t_n = n\Delta t$ with $n = 0, 1, .., N$ and $\Delta t = T/N$

- ## Step 2: Fulfilling the eq. at discrete grid points

  Our governing eq. from Step 0 is rewritten to have a solution at each defined time step.

$$\frac{dC^n}{dt} = -kC^n \qquad \text{With } n = 0, 1, \ldots, N$$

How to formulate a finite difference scheme?...

- Step 3: Replacing derivatives by finite difference

A first approximation of the first derivative is:

$$\frac{dC^n}{dt} = \frac{C^{n+1} - C^n}{\Delta t}$$

- Step 4: Formulating a recursive algorithm

We need a computational algorithm that will provide C^{n+1} as a function of already computed quantities (C^n).

$$\frac{C^{n+1} - C^n}{\Delta t} = -kC^n \quad \Longrightarrow \quad C^{n+1} = (1 - k\Delta t)C^n$$

- Step 5: Implementing the numerical script and visualizing the results

Example 2: Numerical and analytical solutions of first order ODE

Here, we will model the evolution of a chemical compound defined as C with $C(0) = 1$ and $k = 5\text{x}10^{-3}$ [1/s] over a time $t \in [0 : T]$, and $T = 1000$ [s]

- Write a script for calculating and plotting the analytical solution
- Write a script to implement a numerical scheme following Steps 0 to 4
- Compare the analytical and numerical solution

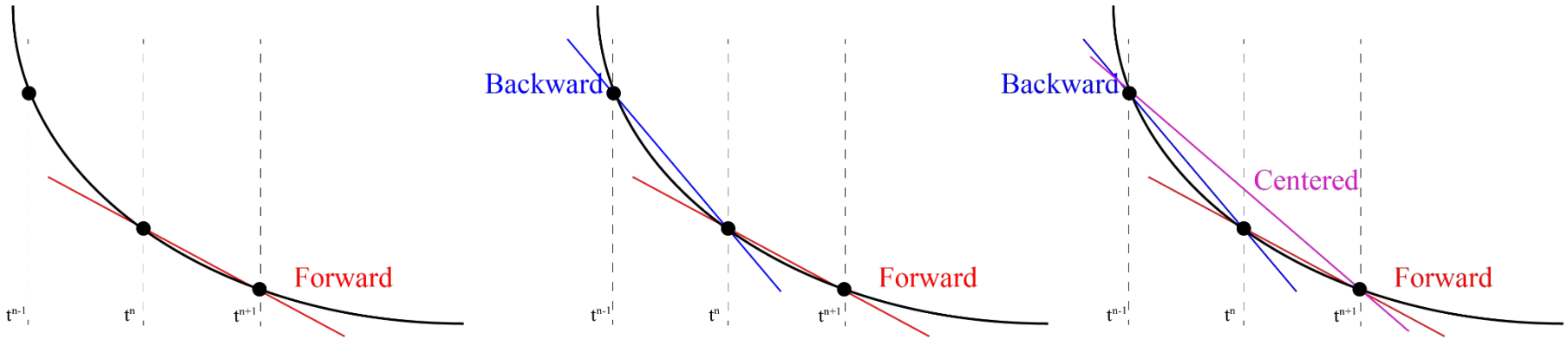
Check the corresponding section in:
Numerical_methods_Part_I.ipynb

Different discretization of the first derivative

- Back to Step 3

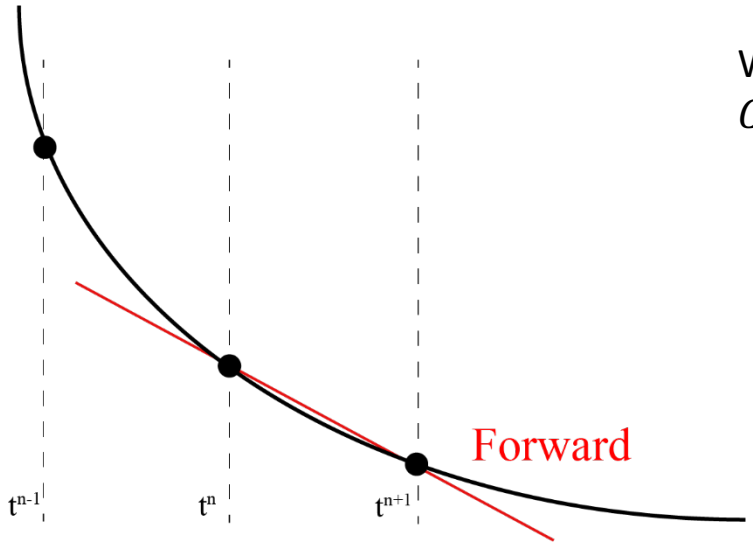
$$\frac{dC^n}{dt} = \frac{C^{n+1} - C^n}{\Delta t}$$

Step 3 can be resolved differently with a vast range of discretization methods.



Three of the most basic numerical discretization methods. There are many more!

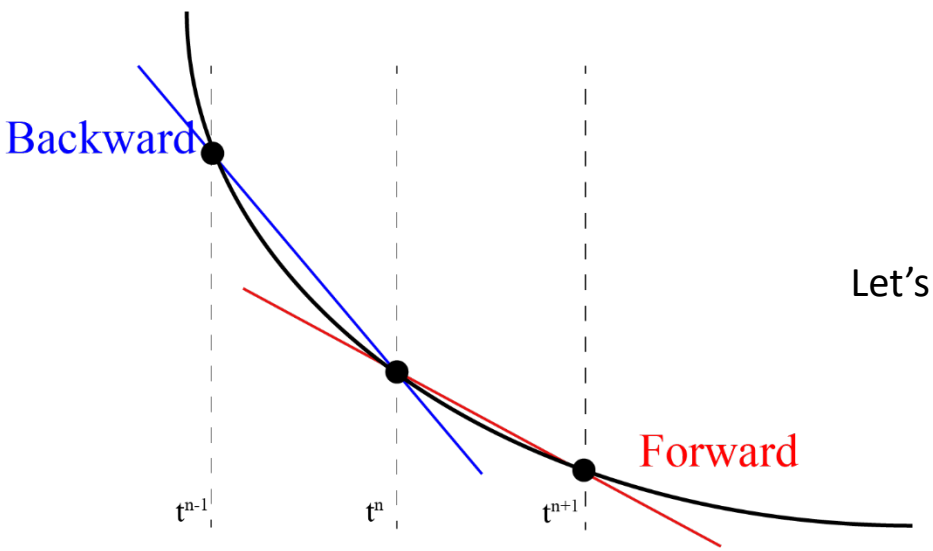
Forward and backward methods



We need a computational algorithm that will provide C^{n+1} as a function of already computed quantities (C^n):

$$\frac{C^{n+1} - C^n}{\Delta t} = -kC^n$$

$$\boxed{C^{n+1} = (1 - k\Delta t)C^n}$$



$$\frac{C^n - C^{n-1}}{\Delta t} = -kC^n$$

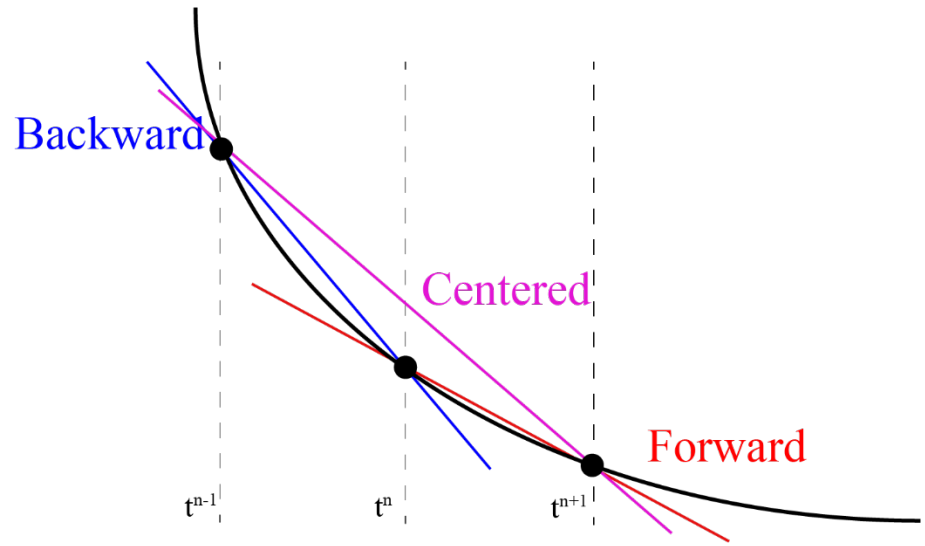
Let's replace *n* by *n+1*:

$$\frac{C^{n+1} - C^n}{\Delta t} = -kC^{n+1}$$

$$\boxed{C^{n+1} = \frac{1}{1 + k\Delta t}C^n}$$

The centered (Crank-Nicolson) method

We previously collected information either backward or forward. We want to use both information. We now look at the derivative at the point $t^{n+\frac{1}{2}} = \frac{1}{2}(t^{n+1} + t^n)$,i.e., $t^{n+\frac{1}{2}} = \left(n + \frac{1}{2}\right)\Delta t$. The approximation at this (fictive) time step is:



$$\frac{dC^{n+\frac{1}{2}}}{dt} \sim \frac{C^{n+1} - C^n}{\Delta t} \quad \Longrightarrow \quad \frac{C^{n+1} - C^n}{\Delta t} = -kC^{n+\frac{1}{2}}$$

Let's express $C^{n+\frac{1}{2}}$ as: $$C^{n+\frac{1}{2}} = \frac{1}{2}(C^{n+1} + C^n)$$

Then we obtain: $$\boxed{C^{n+1} = \frac{1 - \frac{1}{2}k\Delta t}{1 + \frac{1}{2}k\Delta t}C^n}$$

Exercise 1: Forward, backward, and Crank-Nicolson methods

- Modify "forward_solver" function to solve the simple ODE
- Develop a function to solve the simple ODE using the Backward method
- Develop a function to solve the simple ODE using the Crank-Nicolson method
- Show on one single figure the results from the three methods

Check the corresponding section in:
Numerical_methods_Part_I.ipynb

Different discretization of the first derivative

- Back to Step 3

$$C^n = -\frac{1}{k}\frac{dC^n}{dt}$$

Note that in this case the derivative is negative and its absolute value decrease over time. Therefore, using backward scheme overestimates, and forward underestimates.

- Generalization the θ-rule

$$C^{n+1} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t}C^n$$

θ = 0 recovers the Forward method ⇒ $\boxed{C^{n+1} = (1-k\Delta t)C^n}$

θ = 1 recovers the Backward method ⇒ $\boxed{C^{n+1} = \frac{1}{1+k\Delta t}C^n}$

θ = 0.5 recovers the Crank-Nicolson method ⇒ $\boxed{C^{n+1} = \frac{1-\frac{1}{2}k\Delta t}{1+\frac{1}{2}k\Delta t}C^n}$

Exercise 2: Evaluating the effect of time-step

- Graphically evaluate how each model respond to a reduction and an increase of time step

Check the corresponding section in:
Numerical_methods_Part_I.ipynb

- Step 6: Verifying your code

Summary of Exercise 2:

- Backward Euler: monotone solution but needs small time step
- Crank-Nicolson: most accurate solution but solution oscillates for $\Delta t > 400$
- Forward Euler:
  - monotone solution for $\Delta t < 200$
  - decaying oscillation for $\Delta t > 200$
  - growing oscillation for $\Delta t > 400$

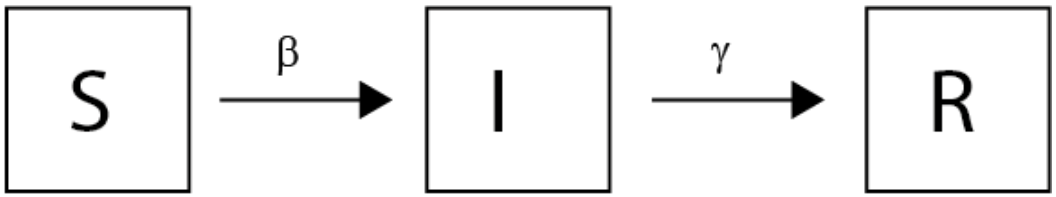
Do we have mathematical criteria to prevent oscillatory solution (given our monotonic analytical solution)?

⇓

See the **"Supplemental Information"** slides below together with some scripts!

- For example: The S-I-R problem

- SIR (Susceptible-Infected-Removed) model of disease spread
- All individuals in the population are assumed to be in one of these three states.
- The states are: susceptible (S), infected (I) and removed (R).



* S = Susceptible population
* I = Infected population
* R = Removed population

Step 0: Turn the question into equations

This is probably the most important step. We have a system of ODE to solve numerically.

$$S \xrightarrow{\beta} I \xrightarrow{\gamma} R$$

$$\frac{\partial S}{\partial t} = -\beta I(t)S(t)$$

$$\frac{\partial I}{\partial t} = \beta I(t)S(t) - \gamma I(t)$$

$$\frac{\partial R}{\partial t} = \gamma I(t)$$

β: transmission rate (the rate at which individuals bump into others and expose them to the virus).

γ: recovery rate (the rate at which those who are infected become immune)

Parameters: We simplify further the problem with β and γ being constant.

First guess: γ = 1/15 [1/days]

An epidemic occurs if the number of infected individuals increases: $\frac{\partial I}{\partial t} > 0$

That is: $\beta I(t)S(t) - \gamma I(t) > 0$

at the beginning S=1, the condition is $\frac{\beta}{\gamma} = R_0 > 1$

We start with R_0=2, that is β= 2/15 (in a more complex model, β would vary with time).

Step 0: Turn the question into equations…

How do we get this?

$$\frac{\partial S}{\partial t} = -\beta(t)I(t)S(t)$$

1- Each susceptible person in S will be in contact with a fraction, p of the infected population I at each time step.

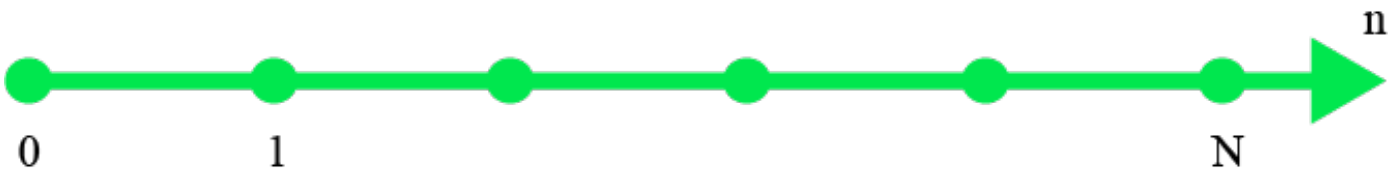
2- We can then count pI x S contacts between the susceptible and infected persons at each time step.

3- However, only a fraction q of the defined contact will propagate the infection: $qpIS$.

4- We define β=qp.

Step 1: Discretizing the domain

We introduce a finite mesh that will approximate the continuous domain. Solution of the model will be expressed at these discrete times.



$t_n = n\Delta t$ with $n = t_{ini}, t_{ini}+\Delta t, .., t_{end}$

Here we take:

- $t_{ini} = 0$
- $t_{end} = 500$ days
- $\Delta t = 1$ day

Steps 2 and 3

- Step 2: Rewrite the eq. at discrete grid points

$$\frac{\partial S^n}{\partial t} = -\beta I^n S^n$$

$$\frac{\partial I^n}{\partial t} = \beta I^n S^n - \gamma I^n$$

$$\frac{\partial R^n}{\partial t} = \gamma I^n$$

with $t_n = n\Delta t$ with $n = t_{ini}, t_{ini}+\Delta t, .., t_{end}$

- Step 3: Replacing derivatives by finite differences

Here we propose to start with the forward method:

$$\frac{dC^n}{dt} = \frac{C^{n+1} - C^n}{\Delta t}$$

Step 4: Formulating a recursive algorithm

We need a computational algorithm that will provide C_{n+1} as a function of already computed quantities (C_n).

$$\frac{S^{n+1} - S^n}{\Delta t} = -\beta I^n S^n$$

$$\frac{I^{n+1} - I^n}{\Delta t} = \beta I^n S^n - \gamma I^n$$

$$\frac{R^{n+1} - R^n}{\Delta t} = \gamma I^n$$

That can be rewritten as:

$$\boxed{\begin{aligned} S^{n+1} &= S^n - \Delta t \beta I^n S^n \\ I^{n+1} &= I^n + \Delta t \beta I^n S^n - \Delta t \gamma I^n \\ R^{n+1} &= R^n + \Delta t \gamma I^n \end{aligned}}$$

Example 3: Solving system of first order differential equations

- Implement the numerical scheme from Steps 0 to 4
- Try to write useful functions to make your scripts reproducible

Check the corresponding section in:
Numerical_methods_Part_I.ipynb

Biogeochemical Modelling of Lakes, Sediments and Oceans
701-1338-00L 2026

Numerical Modeling of Ordinary Differential Equations (ODEs)

Supplemental Information

Aaron Wienkers, awienkers@ethz.ch

Original Slide Credit: Abolfazl Irani Rahaghi

Stability criteria - Von Neumann stability analysis

Goal: to investigate the propagation and amplification of numerical errors.

Any error $\epsilon(t)$ in a solution can be represented by a superposition of oscillating functions:

$$\epsilon(t) = \sum_{m=1}^{M} e^{\alpha_m t}$$

For a given wavenumber, we have:

$$\epsilon^n = e^{\alpha t}$$
$$\epsilon^{n+1} = e^{\alpha(t+\Delta t)}$$

Definition: A numerical scheme is stable when the amplitude of error is damped. This is the case when the norm of the error <1:

$$|\epsilon(\alpha \Delta t)|^2 \leq 1$$

Stability of θ-rule model

Reaction-type equation: $$\frac{dC(t)}{dt} = -kC(t)$$

Algorithm notation: $$C^{n+1} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t} C^n$$

Analysis of a perturbation:

$$\epsilon^{n+1} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t}\epsilon^n$$

$$e^{\alpha(t+\Delta t)} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t} e^{\alpha t}$$

$$e^{\alpha\Delta t} = \frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t}$$

$$\boxed{|e^{\alpha\Delta t}|^2 = \left(\frac{1-(1-\theta)k\Delta t}{1+\theta k\Delta t}\right)^2}$$

Stability of forward, backward, and Crank-Nicolson methods

$\theta = 0$ ⟹ Forward Euler method $|e^{\alpha \Delta t}|^2 = (1 - k\Delta t)^2$

$$|e^{\alpha \Delta t}|^2 \leq 1 \Longrightarrow (1 - k\Delta t)^2 \leq 1 \Longrightarrow \boxed{\Delta t \leq \frac{2}{k}} \Longrightarrow$$

The Forward Euler method is conditionally stable.

$\theta = 1$ ⟹ Backward Euler method $\boxed{|e^{\alpha \Delta t}|^2 = \left(\frac{1}{1 + k\Delta t}\right)^2}$ ⟹ **The Backward Euler method is unconditionally stable.**

$\theta = 0.5$ ⟹ Crank-Nicolson method $|e^{\alpha \Delta t}|^2 = \left(\frac{1 - 0.5k\Delta t}{1 + 0.5k\Delta t}\right)^2$

$$|e^{\alpha \Delta t}|^2 \leq 1 \Longrightarrow \left(\frac{1 - 0.5k\Delta t}{1 + 0.5k\Delta t}\right)^2 \leq 1 \Longrightarrow \boxed{k\Delta t \geq 0} \Longrightarrow$$

The Crank-Nicolson method is unconditionally stable.

Consistency

Criterion: The discretization of an ODE (or a PDE) should become exact as the mesh size tends to zero (truncation error should vanish).

Critically assess your model output

- is the response consistent (monotonic, oscillatory, etc)?
- run simple case that you can either solve manually or analytically
- this step will help debugging codes for **typos** (i.e. most common issue in programming)

Forward Euler method: $$\boxed{C^{n+1} = (1 - k\Delta t)C^n}$$

We want a monotonic function. This can't be the case for $k\Delta t > 1$.

Crank-Nicolson method: $$\boxed{C^{n+1} = \frac{1 - \frac{1}{2}k\Delta t}{1 + \frac{1}{2}k\Delta t}C^n}$$

We want a monotonic function. This can't be the case for $k\Delta t > 2$.

Accuracy of θ-rule model

While **stability** concerns the **qualitative** properties of the numerical solution, it remains to investigate the **quantitative** properties to see exactly how large the numerical errors are.

Error estimate

We define first the exact solution, C_e of the problem and compare the exact solution with the model solution C. The error for a given time step, ΔT is given by:

$$Error = \sqrt{\Delta t \sum_{t=1}^{T} (C_e - C)^2}$$

Convergence

We see that the error, E, decreases with the grid Δt. The relationship follow a power-law. We can show that most numerical methods follow power-law relationship (for small Δt):

$$E = cte(\Delta t)^p$$

where p is called the convergence rate and can be estimated (knowing the analytical solution) by fitting the line on the previous figure or as:

$$p = \frac{ln(E(2\Delta T) - E(\Delta T))}{ln(\Delta T - 2\Delta T)}$$

Lax equivalence theorem:

stability + consistency = convergence

Estimating the accuracy of the model without analytical solution

We can show that:

$$Error = \sqrt{\Delta t \sum_{t=1}^{T} (C_e - C_{\Delta t})^2}$$

Which can also be expressed as:

$$Error = \frac{C_{\Delta t} - C_{2\Delta t}}{2^p - 1}$$

The convergence rate, p, can also be expressed as:

$$p = ln \frac{C_{2\Delta t} - C_{4\Delta t}}{C_{\Delta t} - C_{2\Delta t}}$$

For practical purposes, convergence can be investigated numerically by comparing the results computed on a series of successively refined grids.

Exercise 3: Test the convergence of the Crank-Nicolson method

- Use the "convergence" function above with a few different time steps

Check the corresponding section in:
Numerical_methods_Part_I.ipynb

Order of convergence

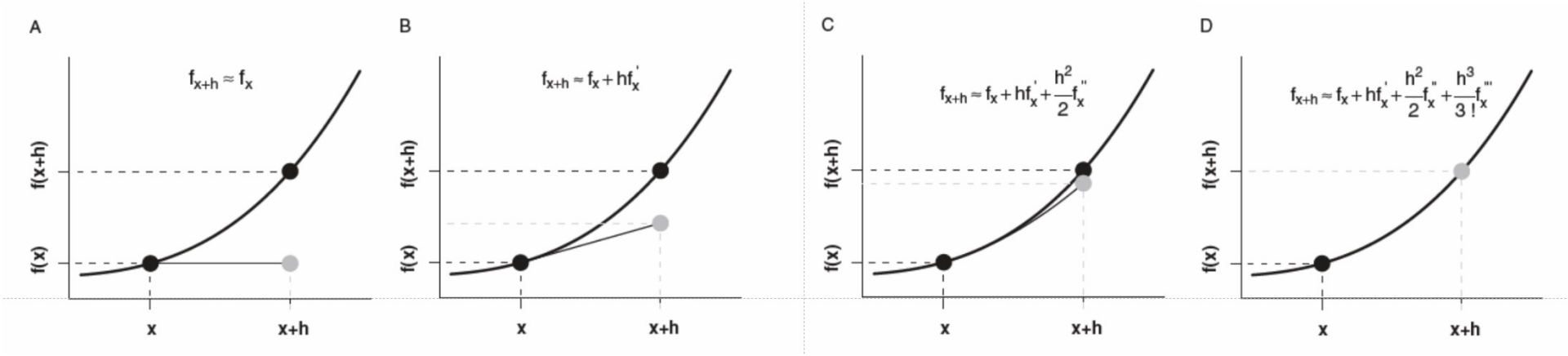
The order of convergence is related to the global **truncation error**.

Taylor series reminder:

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + \ldots + + \frac{(\Delta t)^n}{n!}\frac{d^n f}{dt^n}$$

Taylor series truncator:

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + O((\Delta t)^4))$$



Order of convergence: Forward and backward method

$$f(t_n + \Delta t) = f(t_n) + \frac{\Delta t}{1!}\frac{df}{dt} + \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} + \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} + O((\Delta t)^4))$$

$$\Delta t \frac{df}{dt} = f(t_n + \Delta t) - f(t_n) - \frac{(\Delta t)^2}{2!}\frac{d^2 f}{dt^2} - \frac{(\Delta t)^3}{3!}\frac{d^3 f}{dt^3} - O((\Delta t)^4))$$

$$\frac{df}{dt} = \frac{f(t_n + \Delta t) - f(t_n)}{\Delta t} - \frac{\Delta t}{2!}\frac{d^2 f}{dt^2} - \frac{(\Delta t)^2}{3!}\frac{d^3 f}{dt^3} - O((\Delta t)^3))$$

$$\frac{df}{dt} = \frac{f(t_n + \Delta t) - f(t_n)}{\Delta t} - O(\Delta t)$$

First order